Fundamentación

Linux: Introducción

Sergio A. Cantillo

sacantillo@uao.edu.co

Facultad de Ingeniería y Ciencias Básicas
Universidad Autónoma de Occidente

Agenda

1. Introducción a Linux

Agenda

1. Introducción a Linux
2. Sistema de Archivos

Agenda

1. Introducción a Linux
2. Sistema de Archivos
3. Navegar a través del Sistema de Archivos

Agenda

1. Introducción a Linux
2. Sistema de Archivos
3. Navegar a través del Sistema de Archivos
4. Interactuar con el Sistema de Archivos

Agenda

1. Introducción a Linux
2. Sistema de Archivos
3. Navegar a través del Sistema de Archivos
4. Interactuar con el Sistema de Archivos
5. Permisos

Agenda

1. Introducción a Linux
2. Sistema de Archivos
3. Navegar a través del Sistema de Archivos
4. Interactuar con el Sistema de Archivos
5. Permisos
6. bash scripts

Agenda

1. Introducción a Linux
2. Sistema de Archivos
3. Navegar a través del Sistema de Archivos
4. Interactuar con el Sistema de Archivos
5. Permisos
6. bash scripts
7. Comandos **apt** y **sudo**

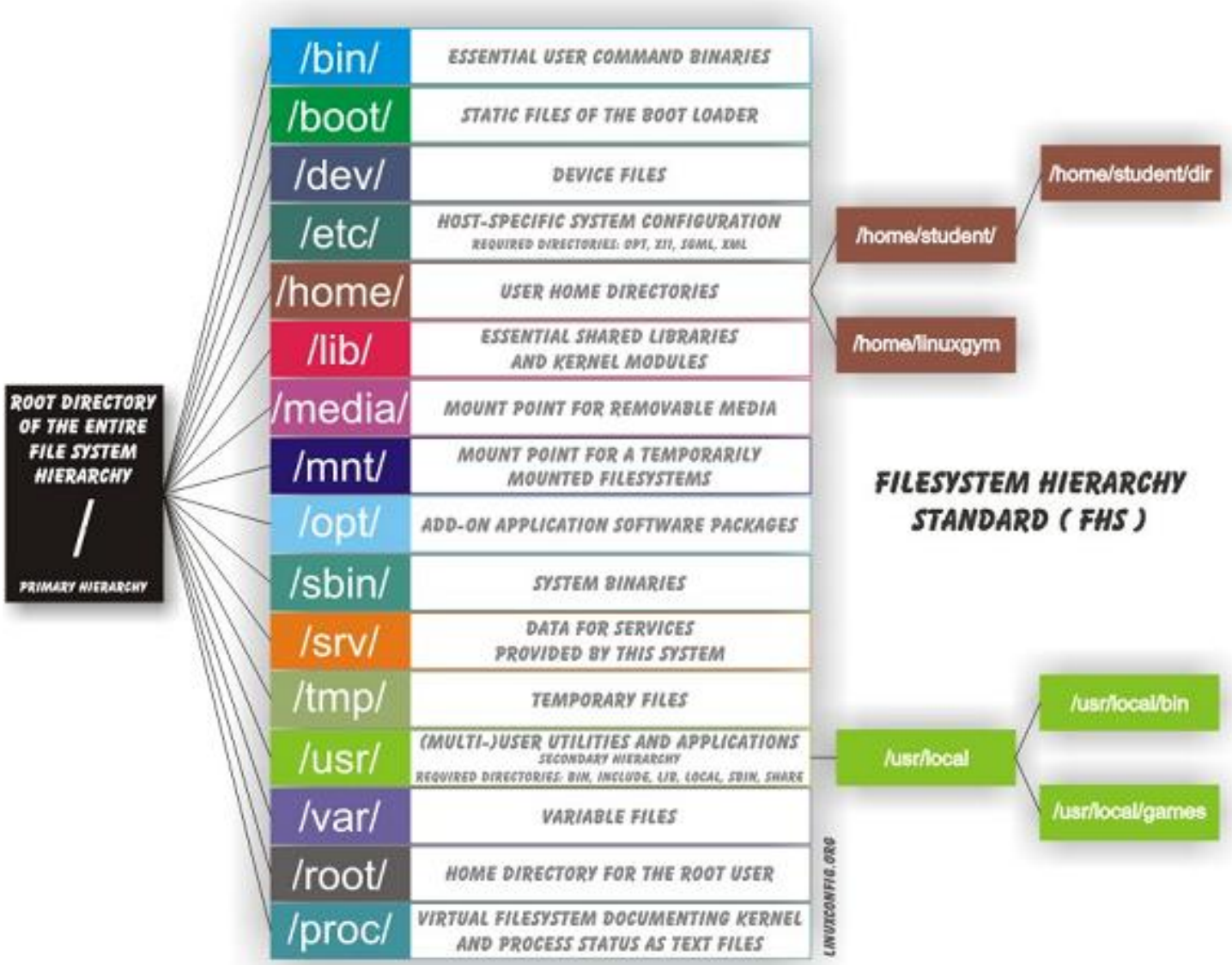
Agenda

1. Introducción a Linux
2. Sistema de Archivos
3. Navegar a través del Sistema de Archivos
4. Interactuar con el Sistema de Archivos
5. Permisos
6. bash scripts
7. Comandos **apt** y **sudo**
8. Referencias

Porque aprender Linux?

- Desarrollo de algoritmos de IA.
- Linux esta en todas partes.
- Versatilidad
- Seguridad

Sistema de Archivos de Linux



Navegar a través del Sistema de Archivos

- Carpetas (directorios)
- Archivos



Acceso Ubuntu Online - `https://www.onworks.net/`

Comando **pwd**

El comando **pwd** permite conocer la ruta(path) actual en el cual nos encontramos al ejecutar el terminal (Ctrl + t).

```
jose@asus-N46VB:~$ pwd
/home/jose
```

Comando **cd**

El comando **cd** permite ir a un directorio específico.

```
cd /home/user/Downloads
```

Para retroceder o volver a un directorio se utiliza el comando:

```
cd ../
```

Para regresar al directorio **/home** del usuario se utiliza el comando:

```
cd ~
```

Comando **ls**

El comando **ls** es usado para conocer(listar) el contenido de un directorio.

```
jose@asus-N46VB:~$ ls
Desktop    MEGA       prueba1   pytorch     tensorflow
Documents  Music      prueba2   snap        tor.keyring
Downloads  Pictures   Public    Templates   Videos
```

El comando **ls -a** permite listar todos los elementos de un directorio incluyendo los archivos **ocultos**

Obetener Informacion acerca de los comandos - **-help**

```
jose@asus-N46VB:~$ ls --help
```

Comando **mkdir**

El comando **mkdir** permite la creación de nuevos directorios.

```
jose@asus-N46VB:~$ mkdir my_folder
```

Comando **touch**

El comando **touch** permite crear o actualizar (si ya existe) nuevos archivos. Es importante tener en cuenta que se debe conocer su respectiva extensión.

```
jose@asus-N46VB:~$ touch my_file.txt
```

Editor Visual **vi**

vi es la herramienta por defecto de Linux para editar archivos.

vi posee dos modos diferentes:

- **Command Mode:** este modo permite usar comandos para interactuar con el archivo.
- **Insert Mode:** este modo permite insertar texto en el archivo.

Command Mode

Por defecto **vi** abre con el **command mode** activado. Para cambiar al **insert mode** se debe pulsar la tecla **i**

Algunos Comandos de **vi**

- **:q** permite salir del archivo.
- **:w** permite escribir (guardar) los cambios en el archivo.
- **:q!** permite salir sin guardar los cambios en el archivo.
- **:help** permite obtener ayuda online.

Comando **mv**

mv permite mover archivos o directorios de una ubicación a otra.

```
mv <archivo/carpeta a mover> <destino>
```

Comando **cp**

cp permite copiar un archivo o una carpeta (o múltiples) desde una ubicación a otra.

```
cp -r <ubicacion carpeta a copiar> <ubicacion de la nueva carpeta>
```

Copiar Carpetas

Para copiar carpetas en vez de archivos se debe usar acompañando el comando con **-r**. En caso de no existir, se crea esta ubicación:

```
cp -r <ubicacion carpeta a copiar> <ubicacion de la nueva carpeta>
```

Comando **rm**

rm permite remover archivos y carpetas de una determinada ubicación.

- para remover archivos usar:

```
rm <file to remove>
```

- para remover forzadamente archivos usar:

```
rm -f <file to remove>
```

- para remover carpetas usar:

```
rm -r <folder to remove>
```

- para remover forzadamente carpetas usar:

```
rm -rf <file to remove>
```

Permisos

- El comando **ls -la** permite ver todos los archivos o carpetas con información acerca de su fecha creación y sus permisos.

Ejemplo

```
-rw-r--r-- 1 user user 2203 Jul 23 23:26 move_bb8_square.py
```

Permisos de archivo o carpeta

cada archivo o carpeta tiene 3 tipos de permisos:

- **read**: hace referencia a que el usuario puede leer el contenido del archivo. Se representa con la letra **r**.
- **write**: hace referencia a que el usuario puede escribir o modificar el archivo. Se representa con la letra **w**.
- **execute**: hace referencia a que el usuario puede ejecutar el archivo. Se representa con la letra **x**.

Permisos de Grupo

- **owner**: aplica solo para el propietario del archivo y no afecta las acciones de otros usuarios. Se representa por las 3 primeras letras.
- **group**: aplica solo para el grupo que ha sido asignado el archivo y no afecta las acciones de otros usuarios. Se representa por las 3 letras intermedias.
- **all users**: aplica para todos los usuarios. Se representa por las 3 ultimas letras.

Comando **chmod**

El comando **chmod** permite modificar los permisos de uno o varios archivos y carpetas.

Estructura del comando

```
chmod <groups to assign the permissions><permissions to assign/remove><file/folder names>
```

Para los grupos, se utilizan las siguientes **flags:**

- **u**:Owner
- **g**:Group
- **o**:Others
- **a**:All users

Comando **chmod**

Ejemplo

1. crear un archivo con nombre **my_script.py**
2. verificar los permisos con el comando **ls -la**
3. modificar los permisos del archivo para que se pueda ejecutar

```
chmod +x my_script.py
```

4. dar permisos de escritura solo al grupo.

```
chmod g+x my_script.py
```

5. eliminar permisos de ejecucion a **group and others**

```
chmod go-x my_script.py
```

Comando **chmod**

Binary References

Binary References es un método alternativo para modificar los permisos usando el comando **chmod**

El metodo consiste en cambiar los caracteres (rwxrwxrwx) por 3 numeros.

- r $= 4$
- w $= 2$
- x $= 1$

luego, estos numero se suman para obtener un numero entero que representa los permisos.

Ejemplo

Establecer los permisos al archivo **my_script.py** de la siguiente manera:

- Todos los permisos al propietario (rwx) $\Rightarrow 4 + 2 + 1 = 7$
- Solo permiso de lectura al grupo (r)$\Rightarrow 4$
- Ningún permiso a todos los demás usuarios $\Rightarrow 0$

Respuesta:

```
chmod 740 my_script.py
```

bash scripts

- **bash** es un lenguaje de ordenes que permite la interaccion del usuario y el sistema operativo Linux.
- Es una interfaz de linea de comandos.
- Permite un mayor grado de precision

bash script

- Es un archivo de texto que contiene una serie de comandos.

Ejemplo

1. Crear un nuevo archivo con nombre **bash_script.sh**.
2. Escribir los siguientes comandos dentro del archivo.

```
#!/bin/bash
echo Hello there, Programmers!
```

3. Ejecutar el archivo.

bash scripts

Parámetros de un bash script

Es posible enviar parametros a un **bash script** con el fin de que el **script** se ejecute de forma diferente dependiendo del valor de los parámetros.

Ejemplo

demo.sh

```
#!/bin/bash

ARG1=$1

if [ $ARG1 == 'forward' ]; then
echo the robot is moving forward

elif [ $ARG1 == 'rotate' ]; then
echo the robot is rotating

elif [ $ARG1 == 'stop' ]; then
echo the robot is stopping
fi
```

El comando de ejecución:

```
sudo ./demo.sh <value of the parameter>
```

Comandos **apt** y **sudo**

- Linux usa el sistema de paquetes **dpkg**. Este sistema provee programas y aplicaciones para instalar.
- **APT(Advance Package Tool)** es una herramienta de linea de comandos que permite interactuar con el sistema de paquetes.

Algunos comandos comunes

```
apt-get update
apt-get upgrade
apt-get install
```

sudo

sudo es una utilidad de sistemas Linux que permite a los usuarios ejecutar un programa o un comando con privilegios (permisos) de usuario **root**. Esto permite temporalmente convertirse en un **super usuario**

```
sudo apt-get update
```

Referencias

https://linuxconfig.org/bash-scripting-tutorial-for-beginners